



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NAME:VIGNESH P

REG.NO:192525013

TOPIC:1

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

CODE:

```
arr = [10, 25, 30, 45, 50]
```

```
key = 30
```

```
found = False
```

```
for i in range(len(arr)):
```

```
    if arr[i] == key:
```

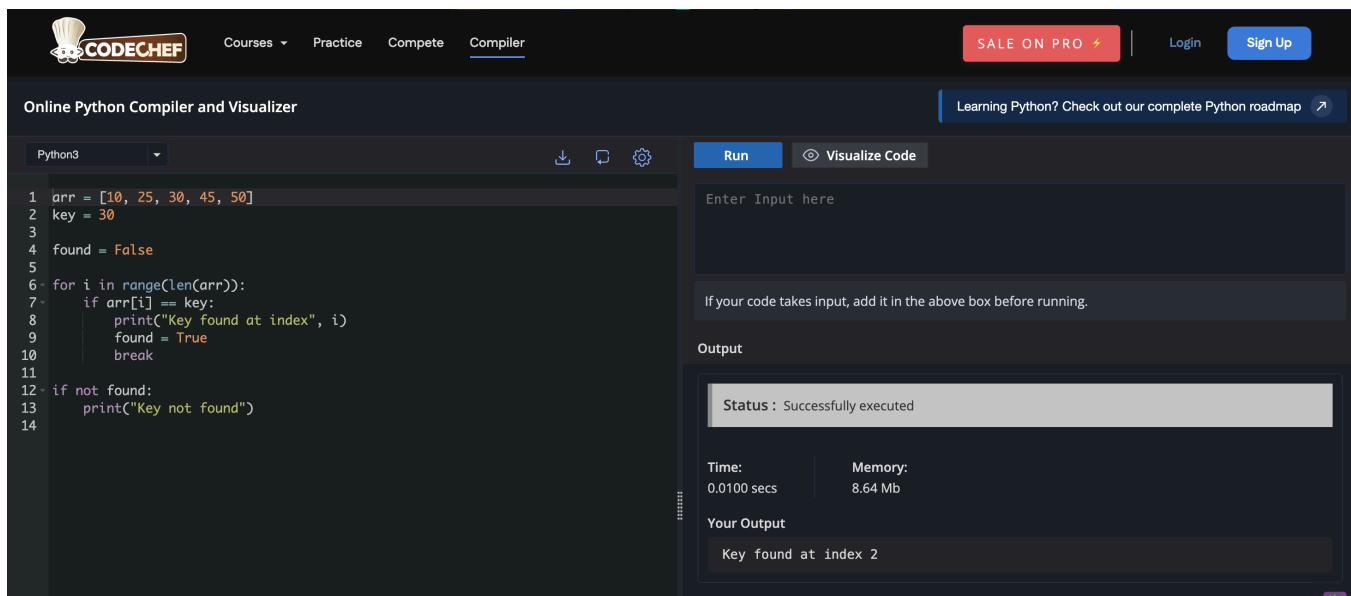
```
        print("Key found at index", i)
```

```
        found = True
```

```
        break
```

```
if not found:
```

```
    print("Key not found")
```



Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω : $O(1)$, O : $O(\log n)$, Θ : $\Theta(\log n)$
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

Binary Search (Recursive)

```
def binary_search(arr, low, high, key):
```

```
    if low <= high:
        mid = (low + high) // 2
```

```
    if arr[mid] == key:
        return mid
```

```

elif arr[mid] > key:
    return binary_search(arr, low, mid - 1, key)

else:
    return binary_search(arr, mid + 1, high, key)

return -1

```

```

arr = [5, 10, 15, 20, 25]
key = 20

```

```

result = binary_search(arr, 0, len(arr) - 1, key)

```

```

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")

```

The screenshot shows an online Python compiler interface. On the left, the code editor contains the following Python code:

```

1 def binary_search(arr, low, high, key):
2
3     if low <= high:
4         mid = (low + high) // 2
5
6         if arr[mid] == key:
7             return mid
8
9         elif arr[mid] > key:
10            return binary_search(arr, low, mid - 1, key)
11
12        else:
13            return binary_search(arr, mid + 1, high, key)
14
15    return -1
16
17
18
19 arr = [5, 10, 15, 20, 25]
20 key = 20
21
22 result = binary_search(arr, 0, len(arr) - 1, key)
23
24 if result != -1:
25     print("Key found at index", result)
26 else:
27     print("Key not found")

```

On the right side of the interface, there are buttons for "Run" and "Visualize Code". Below these, there is an input field labeled "Enter Input here" and a note: "If your code takes input, add it in the above box before running." The "Output" section shows a status message: "Status : Successfully executed". Below this, it displays performance metrics: "Time: 0.0000 secs" and "Memory: 8.936 Mb". The "Your Output" section shows the result: "Key found at index 3".

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.

- **Sample Input:** $n = 5$
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

Factorial - Iterative

```
n = 5
fact = 1

for i in range(1, n + 1):
    fact = fact * i

print("Factorial =", fact)
```

Factorial - Recursive

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print("Recursive Factorial =", factorial(n))
```

The screenshot shows a code editor with a dark theme. The left pane contains the Python code for calculating the factorial of 5 using both iterative and recursive methods. The right-hand panel is divided into sections: 'Enter Input here' (empty), a note about adding input, an 'Output' section showing 'Status : Successfully executed', and performance metrics (Time: 0.0100 secs, Memory: 8.932 Mb). Below these is a 'Your Output' section displaying the results: 'Factorial = 120' and 'Recursive Factorial = 120'.

```
Python3
```

```
1 n = 5
2 fact = 1
3
4 for i in range(1, n + 1):
5     fact = fact * i
6
7 print("Factorial =", fact)
8
9
10
11 def factorial(n):
12     if n == 0 or n == 1:
13         return 1
14     return n * factorial(n - 1)
15
16
17 print("Recursive Factorial =", factorial(n))
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.932 Mb

Your Output

Factorial = 120
Recursive Factorial = 120

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2^n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

Fibonacci Series - Iterative

n = 6

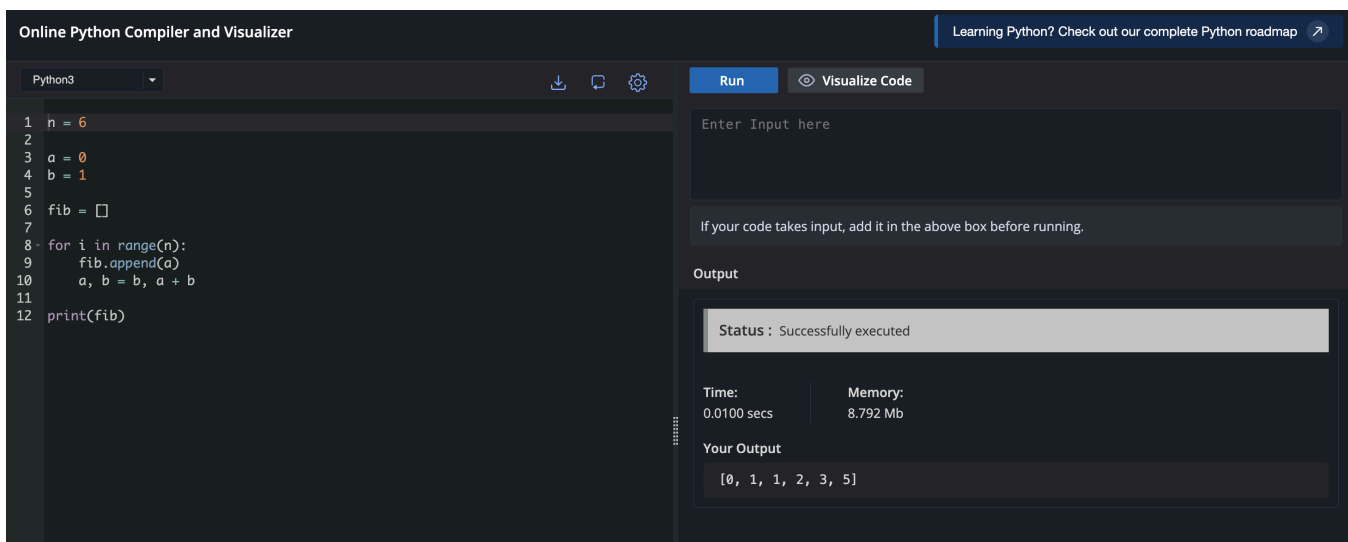
a = 0

b = 1

fib = []

```
for i in range(n):  
    fib.append(a)  
    a, b = b, a + b
```

print(fib)



The screenshot shows an online Python compiler interface. On the left, the code editor contains the following Python code:

```
1 n = 6  
2  
3 a = 0  
4 b = 1  
5  
6 fib = []  
7  
8 for i in range(n):  
9     fib.append(a)  
10    a, b = b, a + b  
11  
12 print(fib)
```

On the right side of the interface, there are buttons for 'Run' and 'Visualize Code'. Below these, there is an input field labeled 'Enter Input here' and a note: 'If your code takes input, add it in the above box before running.' The 'Output' section shows a status message: 'Status : Successfully executed'. Below this, performance metrics are displayed: 'Time: 0.0100 secs' and 'Memory: 8.792 Mb'. Finally, the 'Your Output' section shows the result: '[0, 1, 1, 2, 3, 5]'.

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** A=[[1,2],[3,4]], B=[[5,6],[7,8]]
- **Sample Output:** [[19,22],[43,50]]
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

Matrix Multiplication

```
A = [[1, 2],  
      [3, 4]]
```

```
B = [[5, 6],  
      [7, 8]]
```

```
result = [[0, 0],  
          [0, 0]]
```

```
for i in range(len(A)):  
    for j in range(len(B[0])):  
        for k in range(len(B)):  
            result[i][j] += A[i][k] * B[k][j]
```

```
print(result)
```

The screenshot displays an online Python compiler interface. On the left, a code editor shows the Python code for matrix multiplication. The code defines two 2x2 matrices, A and B, initializes a result matrix, and uses three nested loops to calculate the product. The code is as follows:

```
1  
2 A = [[1, 2],  
3       [3, 4]]  
4  
5 B = [[5, 6],  
6       [7, 8]]  
7  
8 result = [[0, 0],  
9           [0, 0]]  
10  
11 for i in range(len(A)):  
12     for j in range(len(B[0])):  
13         for k in range(len(B)):  
14             result[i][j] += A[i][k] * B[k][j]  
15  
16 print(result)
```

On the right side of the interface, there are buttons for 'Run' and 'Visualize Code'. Below these, an input field is labeled 'Enter Input here'. A message states: 'If your code takes input, add it in the above box before running.' The 'Output' section shows a status message: 'Status : Successfully executed'. Below this, execution statistics are provided: 'Time: 0.0100 secs' and 'Memory: 8.928 Mb'. The 'Your Output' section displays the result of the matrix multiplication: `[[19, 22], [43, 50]]`.

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

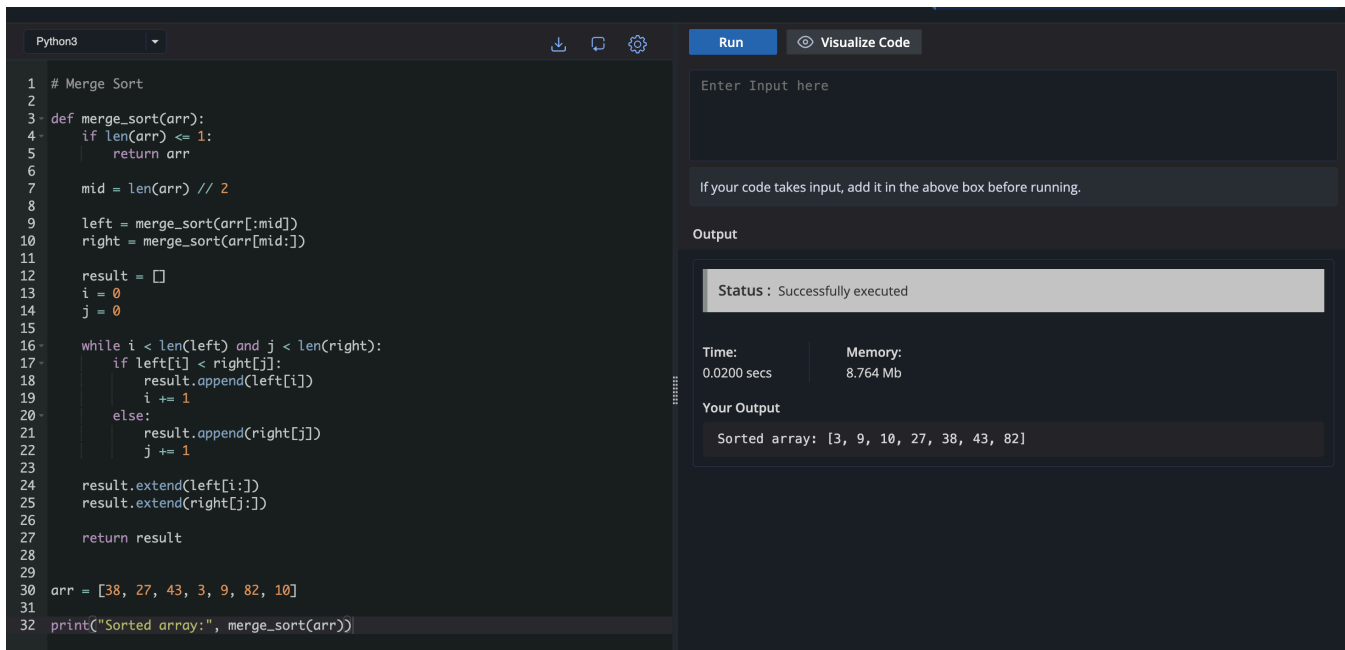
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

```
arr = [38, 27, 43, 3, 9, 82, 10]
```

```
print("Sorted array:", merge_sort(arr))
```



The screenshot shows a code editor with a Python3 interpreter. The code implements a merge sort algorithm. The array [38, 27, 43, 3, 9, 82, 10] is passed to the merge_sort function. The output of the program is "Sorted array: [3, 9, 10, 27, 38, 43, 82]". The execution status is "Successfully executed". The time taken is 0.0200 secs and the memory used is 8.764 Mb.

```
1 # Merge Sort
2
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     mid = len(arr) // 2
8
9     left = merge_sort(arr[:mid])
10    right = merge_sort(arr[mid:])
11
12    result = []
13    i = 0
14    j = 0
15
16    while i < len(left) and j < len(right):
17        if left[i] < right[j]:
18            result.append(left[i])
19            i += 1
20        else:
21            result.append(right[j])
22            j += 1
23
24    result.extend(left[i:])
25    result.extend(right[j:])
26
27    return result
28
29
30 arr = [38, 27, 43, 3, 9, 82, 10]
31
32 print("Sorted array:", merge_sort(arr))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.764 Mb

Your Output

Sorted array: [3, 9, 10, 27, 38, 43, 82]

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log n)$, O : $O(n^2)$, Θ : $\Theta(n \log n)$
- **Explanation:** Shows how pivot choice affects performance.

Quick Sort

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
```

```
    pivot = arr[-1]
```

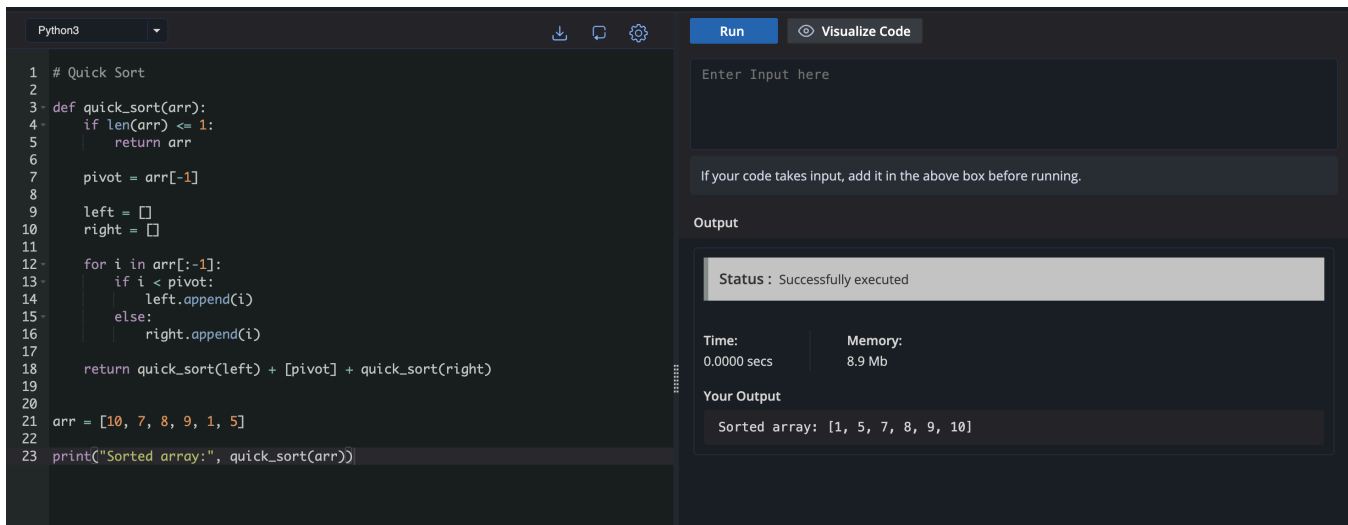
```
left = []  
right = []
```

```
for i in arr[:-1]:  
    if i < pivot:  
        left.append(i)  
    else:  
        right.append(i)
```

```
return quick_sort(left) + [pivot] + quick_sort(right)
```

```
arr = [10, 7, 8, 9, 1, 5]
```

```
print("Sorted array:", quick_sort(arr))
```



The screenshot shows a Python IDE with a dark theme. The left pane contains the following code:

```
1 # Quick Sort  
2  
3 def quick_sort(arr):  
4     if len(arr) <= 1:  
5         return arr  
6  
7     pivot = arr[-1]  
8  
9     left = []  
10    right = []  
11  
12    for i in arr[:-1]:  
13        if i < pivot:  
14            left.append(i)  
15        else:  
16            right.append(i)  
17  
18    return quick_sort(left) + [pivot] + quick_sort(right)  
19  
20  
21 arr = [10, 7, 8, 9, 1, 5]  
22  
23 print("Sorted array:", quick_sort(arr))
```

The right pane has a 'Run' button and a 'Visualize Code' button. Below them is an input field labeled 'Enter Input here' and a note: 'If your code takes input, add it in the above box before running.' The 'Output' section shows a status message: 'Status : Successfully executed'. Below that, a table displays execution metrics:

Time:	Memory:
0.0000 secs	8.9 Mb

Below the table, the 'Your Output' section displays the result: 'Sorted array: [1, 5, 7, 8, 9, 10]'.

Experiment 8: Tower of Hanoi

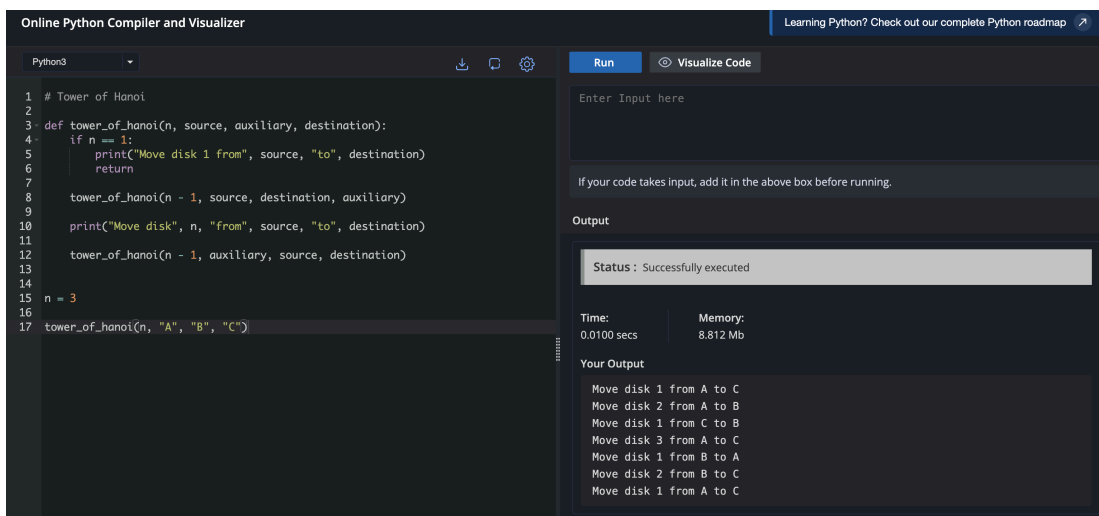
- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$
- **Explanation:** Illustrates exponential growth in recursive calls.

Tower of Hanoi

```
def tower_of_hanoi(n, source, auxiliary, destination):  
    if n == 1:  
        print("Move disk 1 from", source, "to", destination)  
        return  
  
    tower_of_hanoi(n - 1, source, destination, auxiliary)  
  
    print("Move disk", n, "from", source, "to", destination)  
  
    tower_of_hanoi(n - 1, auxiliary, source, destination)
```

n = 3

tower_of_hanoi(n, "A", "B", "C")



The screenshot shows an online Python compiler interface. On the left, the code for the Tower of Hanoi function is displayed, with line numbers 1 through 17. The code defines the function and calls it with n=3 and source 'A', auxiliary 'B', and destination 'C'. On the right, the 'Run' button is highlighted. Below it, the 'Output' section shows the status 'Successfully executed' and the execution time '0.0100 secs' and memory usage '8.812 Mb'. The 'Your Output' section displays the sequence of moves: 'Move disk 1 from A to C', 'Move disk 2 from A to B', 'Move disk 1 from C to B', 'Move disk 3 from A to C', 'Move disk 1 from B to A', 'Move disk 2 from B to C', and 'Move disk 1 from A to C'.

```
1 # Tower of Hanoi  
2  
3 def tower_of_hanoi(n, source, auxiliary, destination):  
4     if n == 1:  
5         print("Move disk 1 from", source, "to", destination)  
6         return  
7  
8     tower_of_hanoi(n - 1, source, destination, auxiliary)  
9  
10    print("Move disk", n, "from", source, "to", destination)  
11  
12    tower_of_hanoi(n - 1, auxiliary, source, destination)  
13  
14  
15 n = 3  
16  
17 tower_of_hanoi(n, "A", "B", "C")
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time: 0.0100 secs Memory: 8.812 Mb

Your Output

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log n)$
- **Explanation:** Shows efficient recursion with logarithmic complexity.

GCD using Euclidean Algorithm

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)
```

a = 48

b = 18

result = gcd(a, b)

print("GCD =", result)

```
Python3  
1 # GCD using Euclidean Algorithm  
2  
3 def gcd(a, b):  
4     if b == 0:  
5         return a  
6     return gcd(b, a % b)  
7  
8  
9 a = 48  
10 b = 18  
11  
12 result = gcd(a, b)  
13  
14 print("GCD =", result)
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.86 Mb

Your Output

GCD = 6

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω : $O(n)$, O : $O(n^2)$, Θ : $\Theta(n^2)$
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

Insertion Sort

```
arr = [12, 11, 13, 5, 6]
```

```
for i in range(1, len(arr)):
```

```
    key = arr[i]
```

```
    j = i - 1
```

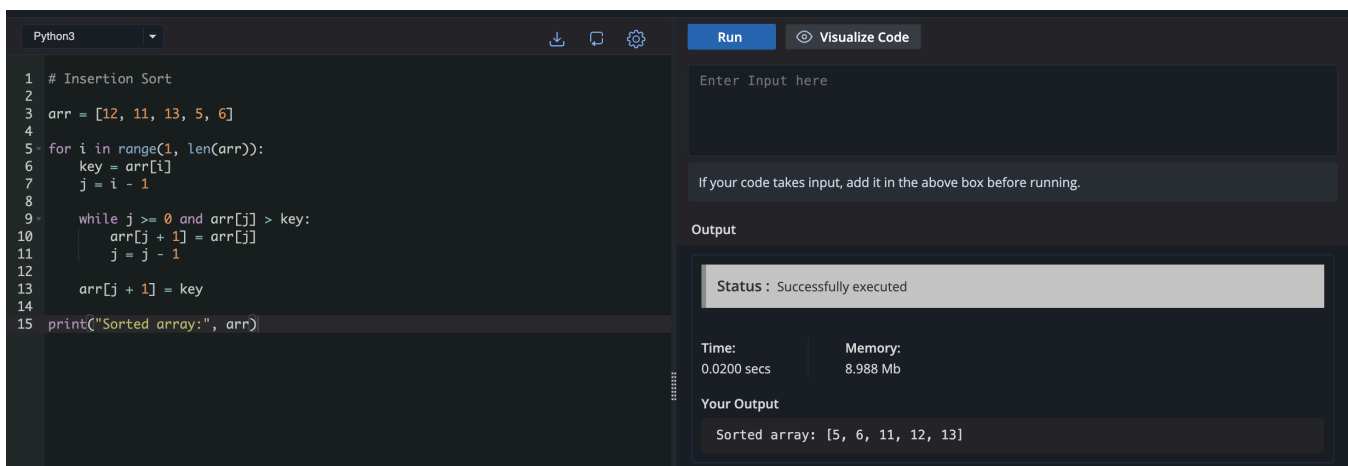
```
    while j >= 0 and arr[j] > key:
```

```
        arr[j + 1] = arr[j]
```

```
        j = j - 1
```

```
    arr[j + 1] = key
```

```
print("Sorted array:", arr)
```



The screenshot shows a Python IDE with the following code in the editor:

```
1 # Insertion Sort
2
3 arr = [12, 11, 13, 5, 6]
4
5 for i in range(1, len(arr)):
6     key = arr[i]
7     j = i - 1
8
9     while j >= 0 and arr[j] > key:
10         arr[j + 1] = arr[j]
11         j = j - 1
12
13     arr[j + 1] = key
14
15 print("Sorted array:", arr)
```

The IDE interface includes a 'Run' button and a 'Visualize Code' button. The output panel on the right shows the following information:

- Enter Input here
- If your code takes input, add it in the above box before running.
- Output
- Status : Successfully executed
- Time: 0.0200 secs
- Memory: 8.988 Mb
- Your Output
- Sorted array: [5, 6, 11, 12, 13]

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Consistent performance using heap structure.

Heap Sort

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
```

```
arr = [4, 10, 3, 5, 1]
```

```
heap_sort(arr)
```

```
print("Sorted array:", arr)
```

Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3

Run

Visualize Code

```
1 # Heap Sort
2
3 def heapify(arr, n, i):
4     largest = i
5     left = 2 * i + 1
6     right = 2 * i + 2
7
8     if left < n and arr[left] > arr[largest]:
9         largest = left
10
11    if right < n and arr[right] > arr[largest]:
12        largest = right
13
14    if largest != i:
15        arr[i], arr[largest] = arr[largest], arr[i]
16        heapify(arr, n, largest)
17
18
19 def heap_sort(arr):
20     n = len(arr)
21
22     # Build max heap
23     for i in range(n // 2 - 1, -1, -1):
24         heapify(arr, n, i)
25
26     # Extract elements
27     for i in range(n - 1, 0, -1):
28         arr[0], arr[i] = arr[i], arr[0]
29         heapify(arr, i, 0)
30
31
32 arr = [4, 10, 3, 5, 1]
33
34 heap_sort(arr)
35
36 print("Sorted array:", arr)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.892 Mb

Your Output

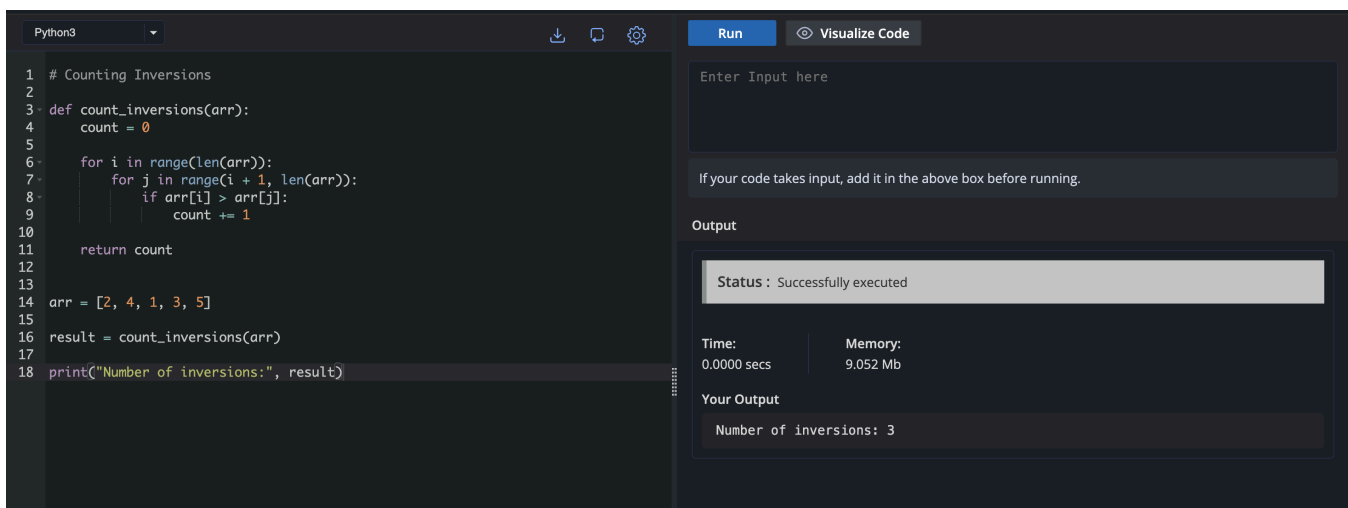
Sorted array: [1, 3, 4, 5, 10]

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

Counting Inversions

```
def count_inversions(arr):  
    count = 0  
  
    for i in range(len(arr)):  
        for j in range(i + 1, len(arr)):  
            if arr[i] > arr[j]:  
                count += 1  
  
    return count  
  
arr = [2, 4, 1, 3, 5]  
  
result = count_inversions(arr)  
  
print("Number of inversions:", result)
```



The screenshot shows a code editor with the following code:

```
1 # Counting Inversions  
2  
3 def count_inversions(arr):  
4     count = 0  
5  
6     for i in range(len(arr)):  
7         for j in range(i + 1, len(arr)):  
8             if arr[i] > arr[j]:  
9                 count += 1  
10  
11     return count  
12  
13  
14 arr = [2, 4, 1, 3, 5]  
15  
16 result = count_inversions(arr)  
17  
18 print("Number of inversions:", result)
```

The editor has a "Run" button and a "Visualize Code" button. The output panel shows the following results:

Status: Successfully executed

Time:	Memory:
0.0000 secs	9.052 Mb

Your Output

```
Number of inversions: 3
```

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

Maximum Subarray Sum using Kadane's Algorithm

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
```

```
max_sum = arr[0]
```

```
current_sum = arr[0]
```

```
for i in range(1, len(arr)):
```

```
    current_sum = max(arr[i], current_sum + arr[i])
```

```
    max_sum = max(max_sum, current_sum)
```

```
print("Maximum subarray sum:", max_sum)
```

```
1 # Maximum Subarray Sum using Kadane's Algorithm
2
3 arr = [-2, -3, 4, -1, -2, 1, 5, -3]
4
5 max_sum = arr[0]
6 current_sum = arr[0]
7
8 for i in range(1, len(arr)):
9     current_sum = max(arr[i], current_sum + arr[i])
10    max_sum = max(max_sum, current_sum)
11
12 print("Maximum subarray sum:", max_sum)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

8.576 Mb

Your Output

Maximum subarray sum: 7

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$
- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log n)$
- **Explanation:** Shows how recursion reduces complexity.

Exponentiation - Iterative

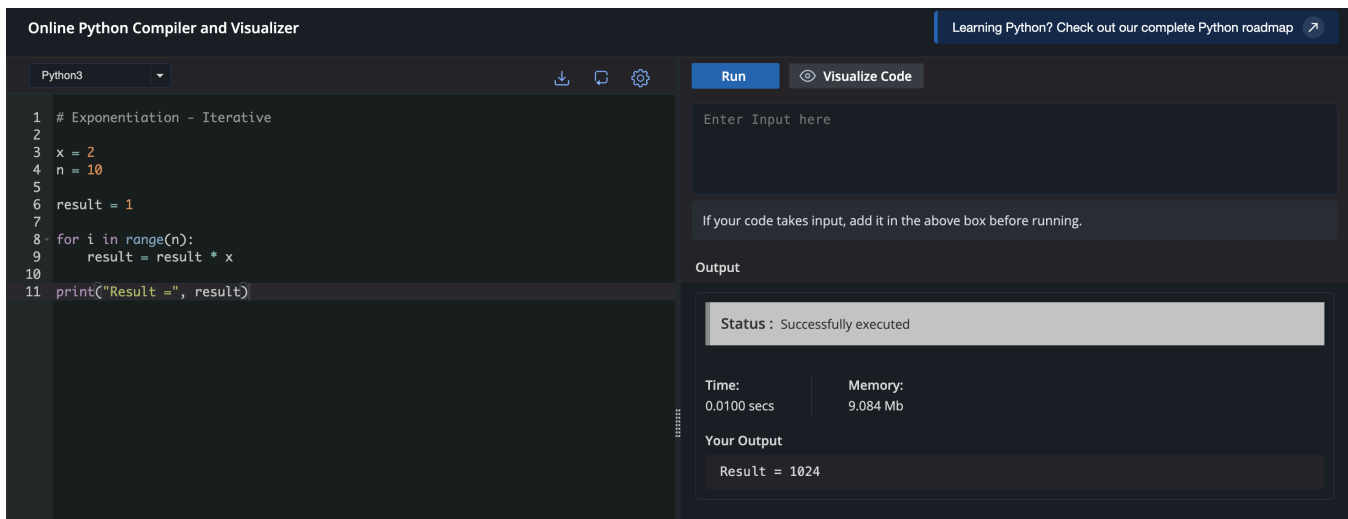
```
x = 2
```

```
n = 10
```

```
result = 1
```

```
for i in range(n):  
    result = result * x
```

```
print("Result =", result)
```



The screenshot shows an online Python compiler interface. The code editor on the left contains the following Python code:

```
1 # Exponentiation - Iterative  
2  
3 x = 2  
4 n = 10  
5  
6 result = 1  
7  
8 for i in range(n):  
9     result = result * x  
10  
11 print("Result =", result)
```

The right panel shows the execution results:

- Run** button is highlighted.
- Enter Input here** text box is empty.
- If your code takes input, add it in the above box before running.** instruction.
- Output** section shows:
 - Status :** Successfully executed
 - Time:** 0.0100 secs
 - Memory:** 9.084 Mb
 - Your Output** section displays: Result = 1024

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)-(3,1), Distance = $\sqrt{5}$
- **Complexity:** Brute force $O(n^2)$, Optimized $O(n \log n)$
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

Closest Pair of Points

```
import math
```

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

```
min_distance = float('inf')
```

```
closest_pair = None
```

```
for i in range(len(points)):
```

```
    for j in range(i + 1, len(points)):
```

```
        x1, y1 = points[i]
```

```
        x2, y2 = points[j]
```

```
        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
```

```
        if distance < min_distance:
```

```
            min_distance = distance
```

```
            closest_pair = (points[i], points[j])
```

```
print("Closest pair:", closest_pair)
```

```
print("Distance =", math.sqrt(5))
```

Python3



Run

Visualize Code

```
1 # Closest Pair of Points
2
3 import math
4
5 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
6
7 min_distance = float('inf')
8 closest_pair = None
9
10 for i in range(len(points)):
11     for j in range(i + 1, len(points)):
12
13         x1, y1 = points[i]
14         x2, y2 = points[j]
15
16         distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
17
18         if distance < min_distance:
19             min_distance = distance
20             closest_pair = (points[i], points[j])
21
22 print("Closest pair:", closest_pair)
23 print("Distance =", math.sqrt(5))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

9.016 Mb

Your Output

```
Closest pair: ((1, 2), (3, 1))
Distance = 2.23606797749979
```